

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

РУБЕЖНЫЙ КОНТРОЛЬ 1

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-34Б _____ (Квиникадзе Б.А.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата защиты:

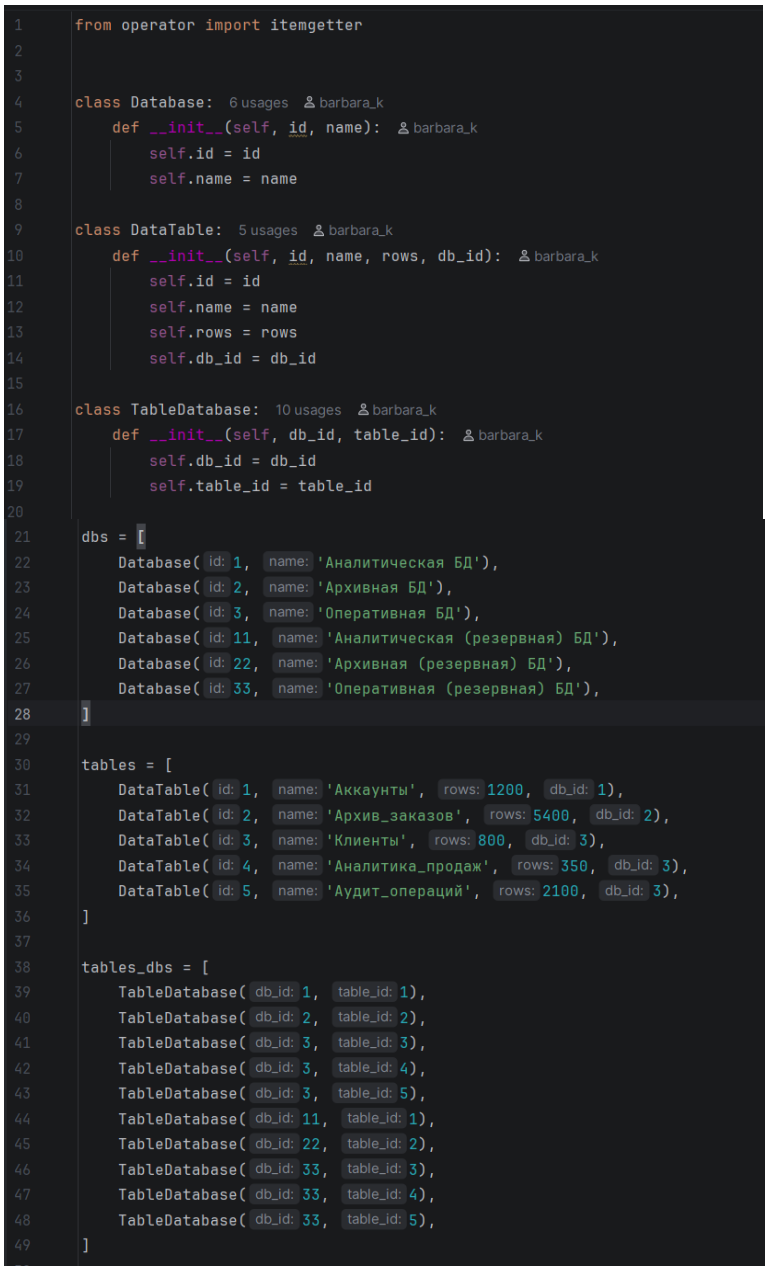
Результаты сдачи (защиты):
- Балльная оценка:

2026 г.

Вариант запросов В

Предметная область 31 – База данных и Таблица данных

Классы: Database хранит данные о базе данных, а DataTable — о таблице данных. Класс TableDatabase хранит идентификаторы таблицы и базы данных для связи «многие ко многим». Классы не содержат логики и используют декоратор @dataclass.
Связь «один ко многим» реализована через поле db_id класса DataTable: одной базе данных соответствует несколько таблиц. Связь «многие ко многим» реализована отдельным списком объектов TableDatabase: каждая таблица может относиться к нескольким базам данных, а каждая база данных — содержать несколько таблиц. Количественный признак — число записей (строк) в таблице.



```python
from operator import itemgetter


class Database:  6 usages  barbara_k
    def __init__(self, id, name):  barbara_k
        self.id = id
        self.name = name

class DataTable:  5 usages  barbara_k
    def __init__(self, id, name, rows, db_id):  barbara_k
        self.id = id
        self.name = name
        self.rows = rows
        self.db_id = db_id

class TableDatabase:  10 usages  barbara_k
    def __init__(self, db_id, table_id):  barbara_k
        self.db_id = db_id
        self.table_id = table_id

dbs = [
    Database( id: 1,  name: 'Аналитическая БД'),
    Database( id: 2,  name: 'Архивная БД'),
    Database( id: 3,  name: 'Оперативная БД'),
    Database( id: 11,  name: 'Аналитическая (резервная) БД'),
    Database( id: 22,  name: 'Архивная (резервная) БД'),
    Database( id: 33,  name: 'Оперативная (резервная) БД'),
]

tables = [
    DataTable( id: 1,  name: 'Аккаунты',  rows: 1200,  db_id: 1),
    DataTable( id: 2,  name: 'Архив_заказов',  rows: 5400,  db_id: 2),
    DataTable( id: 3,  name: 'Клиенты',  rows: 800,  db_id: 3),
    DataTable( id: 4,  name: 'Аналитика_продаж',  rows: 350,  db_id: 3),
    DataTable( id: 5,  name: 'Аудит_операций',  rows: 2100,  db_id: 3),
]

tables_dbs = [
    TableDatabase( db_id: 1,  table_id: 1),
    TableDatabase( db_id: 2,  table_id: 2),
    TableDatabase( db_id: 3,  table_id: 3),
    TableDatabase( db_id: 3,  table_id: 4),
    TableDatabase( db_id: 3,  table_id: 5),
    TableDatabase( db_id: 11,  table_id: 1),
    TableDatabase( db_id: 22,  table_id: 2),
    TableDatabase( db_id: 33,  table_id: 3),
    TableDatabase( db_id: 33,  table_id: 4),
    TableDatabase( db_id: 33,  table_id: 5),
]
```

```python
def main():  1 usage  barbara_k
    one_to_many = [(t.name, t.rows, d.name)
                   for d in dbs
                   for t in tables
                   if t.db_id == d.id]

    many_to_many_temp = [(d.name, td.db_id, td.table_id)
                         for d in dbs
                         for td in tables_dbs
                         if d.id == td.db_id]

    many_to_many = [(t.name, t.rows, db_name)
                    for db_name, db_id, table_id in many_to_many_temp
                    for t in tables if t.id == table_id]


    print('Задание B1')
    res_1 = [(name, db) for name, _, db in one_to_many
             if name.startswith('A')]
    for name, db in res_1:
        print(f'  таблица «{name}» — БД «{db}»')

    print('\nЗадание B2')
    res_2_unsorted = []
    for d in dbs:
        d_tables = list(filter(lambda i: i[2] == d.name, one_to_many))
        if d_tables:
            d_rows = [rows for _, rows, _ in d_tables]
            d_min = min(d_rows)
            res_2_unsorted.append((d.name, d_min))
    res_2 = sorted(res_2_unsorted, key=itemgetter(1))
    for name, mn in res_2:
        print(f'  БД «{name}»: минимальное число записей = {mn}')

    print('\nЗадание B3')
    res_3 = sorted(many_to_many, key=itemgetter(0))
    for name, _, db in res_3:
        print(f'  таблица «{name}» — БД «{db}»')


if __name__ == '__main__':
    main()
```

Рис. 1-4 - классы и определение связей

```
Run  main
C:\Users\kvini\PycharmProjects\RK1_PCPL\.venv\Scripts\python.exe C:\Users\kvini\PycharmProjects\RK1_PCPL\main.py
Задание B1
  таблица «Аккаунты» — БД «Аналитическая БД»
  таблица «Архив_заказов» — БД «Архивная БД»
  таблица «Аналитика_продаж» — БД «Оперативная БД»
  таблица «Аудит_операций» — БД «Оперативная БД»

Задание B2
  БД «Оперативная БД»: минимальное число записей = 350
  БД «Аналитическая БД»: минимальное число записей = 1200
  БД «Архивная БД»: минимальное число записей = 5400

Задание B3
  таблица «Аккаунты» — БД «Аналитическая БД»
  таблица «Аккаунты» — БД «Аналитическая (резервная) БД»
  таблица «Аналитика_продаж» — БД «Оперативная БД»
  таблица «Аналитика_продаж» — БД «Оперативная (резервная) БД»
  таблица «Архив_заказов» — БД «Архивная БД»
  таблица «Архив_заказов» — БД «Архивная (резервная) БД»
  таблица «Аудит_операций» — БД «Оперативная БД»
  таблица «Аудит_операций» — БД «Оперативная (резервная) БД»
  таблица «Клиенты» — БД «Оперативная БД»
  таблица «Клиенты» — БД «Оперативная (резервная) БД»
```

Рис. 5 - результат выполнения программы